SCCM6220 Machine Learning

Week 3 Lab Sheet: MLP and Solver Implementation

Week 2 of this module covered the multilayer perceptron (MLP) and its solution strategy, which allows us to find suitable weight values for a training dataset. In this lab, we will write our own Python code to implement MLP and its solver using Numpy. We will also need to compute the derivatives of the activation functions.

Tasks are colour-coded with badges to indicate the type of task as follows:

Core material that you should complete within the lab.

Practice material for reinforcement.

Extension work for further interest.

Exercise 1: Derivatives of Core Activation Functions

Goal: Become familiar with differentiation, focusing on the most commonly-used activation functions, and build a library to be used in Exercise 2 and later.

We have seen in lectures that to solve models, we need to be able to calculate and use the derivatives of our activation functions. Last week, we created an activation functions python file and we have seen some derivations of the activation derivatives in lectures.

In this exercise, come back to your "activations.py" file and update it to include the derivatives of the following activation functions:

- Heaviside Step
- ReLU
- Sigmoid
- Hyperbolic Tangent (tanh)
- Sigmoid Linear Unit (SiLU)

Test to see if your library works by importing it into an iPython notebook. Visualise the activation functions and their derivatives by creating a vector x with 1000 linearly spaced values between -5 and 5, and using this to plot graphs of your functions. Can you see the relationship between the functions and their derivatives?

Exercise 2: Implementing MLP and its Solution Strategy

Goal: Understand the solution strategy for the multilayer perceptron and build a Python function to implement it.

In this exercise, we will implement from scratch an MLP with 1 hidden layer having n nodes in the hidden layer. This MLP will be used to solve the XOR problem that Perceptron could not solve, so you will only need one output node, but you can extend it to multiple outputs later if you prefer.

1. First, you will need to create a loss function. It would be useful to start building a collection of loss functions in a .py file called, for example, "lossFunctions.py". Add a definition to this file for a loss function called "MSE_loss", given by

$$L(y, \hat{y}) = \frac{1}{n_s} \sum_{i=1}^{n_s} (y_i - \hat{y}_i)^2$$

where n_s is the number of samples; $y_1, \ldots y_{n_s}$ are the ground truth labels of the samples; and $\widehat{y_1}, \ldots \widehat{y_{n_s}}$ are the predicted labels. This should take the ground truth label y and predicted label $\hat{y}$ as input and output the mean squared error.

2. Next, you will need to initialize the training by specifying:
    a. The number of input features, or you could determine this using your training data.
    b. The number of nodes that you want to include in your hidden layer.
    c. The number of outputs; use 1 for binary classification or n for non-binary classification with n classes.
    d. The learning rate
    e. The maximum number of iterations
    f. The set of weights $w_{i,j}^1$ going from your input layer to your hidden layer. You can initialize these as random values and store them in a matrix (array) of size $m \times n$ where m is the number of input nodes, n is the number of hidden nodes.
    g. The set of weights $w_{i,j}^2$ going from your hidden layer to your output layer. You can initialize these as random values and store them in a matrix (array) of size $m \times n$ where m is the number of hidden nodes, n is the number of output nodes.
3. Now, start a loop, which should run to your maximum number of iterations.
    a. Make the forward pass of your input data, storing the linear combination values and activated values (those achieved from using the activation function) in variables for use later.
    b. Calculate the loss.
    c. Determine the weights leading to the output layer using the equations from the lecture.
    d. Determine the weights leading to the hidden layer using the equations from the lecture.
4. Finally, write a code for making a prediction from your testing data. This should be the same as making a forward pass through the model but using your new updated weights.

Exercise 3: Derivatives of Additional Activation Functions

Goal: Practice differentiation and build a more complete set of activation functions and derivatives.

In this exercise, you are asked to derive and add the function derivatives for the remaining activation functions in your library.

Create plots of your new activation functions similarly to Exercise 1 and compare the functions with their derivatives.

Exercise 4: Create a Model Class

Goal: Explore building a more complete Python function, including classes.

Rewrite your Perceptron as a class with fit and predict functions. The inputs for the class should be the learning rate and number of iterations. You can use a fixed initialization of zeros for the weights and bias.

Try training your perceptron with your training data. You will first need to initialize the model by calling the class, and then use the .fit function to train the model and the .predict function to make a prediction on your testing data.